# Document 10 — Agent Workflows → Project 3

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

## Prerequisites

09_langgraph

## How to Read & Practice This Document

- **What:** putting tools, RAG, branching, and saved state together into one working agent.
- **Why:** this is what "building an agent" actually looks like in real work — combining what you already know, not new theory.
- **When:** this is the shape of most real single-agent production systems you'll run into at a job.
- **How to practice:** 1. Skim the reading list — you already know the ideas, you're just combining them now. 2. Add pieces one at a time (**Basic** → **Intermediate**): search as a plain tool first, then branching. Don't connect everything at once. 3. Do the **Real-world/Edge case** exercises with the full graph actually running, start to finish. 4. Do the **Failure** exercise on purpose — a search failing in the middle of a run shouldn't lose saved state. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud, step by step, what happens when search finds nothing useful. If you can't, you're not done.

**Jump to:** Core Concepts · Practice Exercises · Build Task

## The Story — what this document is actually building

Picture the agent you've been building since Doc06 as a person starting their first real job. Doc06 taught it to use one tool. Doc07 taught it to keep trying and stop when it's actually done. Doc08 gave it a filing cabinet full of your documents it can search. Doc09 taught it to make decisions as a real graph — with nodes and conditional edges instead of a hidden `while` loop — and to remember where it left off if it gets interrupted. Every one of those was practiced alone, in its own little exercise, disconnected from the rest.

Document 10 is the day all of that has to work together at once, on a real task. A request comes in. The agent has to decide, on its own, whether this request even needs the filing cabinet — searching when you don't need to wastes time and money, and can even hand the model confusing text that

pulls it off track. If it does need to search, it searches, and whatever it finds has to travel with the task, not get lost — because a few steps later, the agent is going to pause and show a human what it found, and ask "should I go ahead?" before doing anything that can't be undone. That pause is the whole point of Doc09's saved-state trick: the agent can stop completely, wait for a real person to say yes or no — seconds, minutes, or a lunch break later — and then pick up again exactly where it left off, with nothing lost.

That's the whole story: **routing** decides if search is needed. **Search** (from Doc08) finds grounding text when it is. **State** (from Doc09) carries that found text forward so nothing has to be re-searched or re-explained. And an **interrupt** (also Doc09) makes the agent stop and ask before it acts. Put together, this is what a real, single-agent production system looks like — and it's exactly what the Build Task below, **Project 3**, asks you to build: one LangGraph agent that searches your documents when it needs to, pauses for your approval before acting, and finishes with an answer you can actually trust, because you can see exactly what it found and what it did with it.

## Core Concepts (read this first — everything you need is here)

**Topics on this page:** Search as a routed step, not something that always runs · Why "agentic RAG" is really a graph problem, not a search problem · State must carry what was found, not just what was asked · Multi-step search: when one search isn't enough · Combining retrieval with tool calls · Caching search results within one run · When human approval should block vs. just notify

This document doesn't introduce a single new mechanism. Every tool below — nodes, conditional edges, state, checkpointers, `interrupt()` — is exactly what Doc09 already taught you. The actual skill here is combining Docs 06-09 (tools, agents, RAG, graphs) into one working system that decides things on its own: whether to search, whether to search again, and whether to stop and ask a human before acting. A quick shared vocabulary, since this page leans on all four of those documents at once:

| Word | Plain meaning | Where you learned it |
|---|---|---|
| **node** | One Python function that is one step of the graph. It reads state and returns the fields it changed. | Doc09 |
| **conditional edge** | A small function that looks at state and returns the *name* of the next node — how a graph makes a choice. | Doc09 |
| **state** | One typed dictionary that travels through every node — the graph's memory for this run. | Doc09 |
| **checkpointer** | Saves the state after every step, so a paused run can continue later. | Doc09 |
| **`interrupt()`** | Stops the graph inside a node and waits for a human. Resumed with `Command(resume=...)`. | Doc09 |

| Word | Plain meaning | Where you learned it |
|---|---|---|
| **retriever / `retrieve()`** | A function that takes a question and returns the most related chunks of your documents. | Doc08 |
| **tool** | A normal function the model is allowed to call, described by its name, type hints and docstring. | Doc06 |

## Search as a routed step, not something that always runs

A conditional edge is the mechanism — Doc09's Nodes and edges, including conditional edges — and deciding *when to search* is the first real decision this document builds with it. Picture a large government office with one very busy records room. A visitor could always be sent there to pull a file, but most visitors just want directions, or already know the answer. A receptionist who sends everyone to records wastes the clerk's time and makes everyone wait longer, including the people who genuinely needed a file. The router placed right after a task arrives is that receptionist: a small, cheap check deciding whether this one task actually needs your documents at all, before the expensive retrieve step ever runs.

### How it really works

- **What it is:** a conditional edge, placed right after `START` (or after a light "understand the task" node), that returns the name of the next node — `"retrieve"` or straight to `"answer"`.
- **Why it exists:** an embedding call plus a vector-store query cost real money and real time on every single run, and Doc08's "Lost in the middle" fact means unrelated retrieved text sitting in the prompt can actively pull the model off track, not just waste tokens. Search "just in case" is a real cost with its own risk, not a free safety margin.
- **When to reach for it:** the moment your real traffic is a genuine mix of "needs my documents" and "doesn't." If every task in your system always needs the documents (an HR-policy bot where every question is about policy), a router adds a new way to be wrong for no benefit — see the decision table below.
- There are three real ways to write the routing check, cheapest to most flexible, and choosing between them is a judgment call worth making on purpose:
- **Rules (keyword/pattern matching). What:** plain Python — `if "policy" in task.lower(): return "retrieve"`. **Why:** free, instant, and fully predictable — you can read the exact set of phrasings it will catch. **When:** early in a project, or when the topics needing documents are a small, stable, known set of words (a policy bot, a product-manual bot).
- **A small, cheap model call. What:** a fast, cheap model returns a structured `needs_search: bool` (Doc04's structured output). **Why:** real user wording varies more than any keyword list can keep up with, but you still want a fast, cheap, *structured* answer, not free-text parsing. **When:** mixed traffic where the same need is phrased many different ways, and a wrong call is recoverable, not catastrophic.

- **Let the main model decide, as a tool. What:** search itself becomes a tool (Doc06) bound alongside your other tools; the model calls it or doesn't, inside the same loop Doc07 already runs. **Why:** zero extra routing code, and the model can decide to search partway through a task, not only once at the very start. **When:** an already-agentic, multi-tool system, where forcing one up-front yes/no before the model has even started reasoning would be artificial — see Combining retrieval with tool calls below for the full code.
- The routing check itself must stay cheap. Using the same expensive model you use for the real answer to *also* decide whether to search means paying for reasoning twice on every single task — once to decide, once to actually answer.
- `add_conditional_edges(START, route_after_start, ["retrieve", "answer"])` — passing the explicit list of destination names lets LangGraph validate every one of them at `compile()`, the same "free bug-catching" Doc09's graph topic describes. A typo'd node name here fails loudly at compile time, not silently three weeks later.
- The dangerous failure mode is not a crash. A router that wrongly skips search still lets the graph finish, and the model answers confidently from its own general training knowledge instead of your documents — a plausible-looking, silently wrong answer, not an error anyone notices without checking.
- When a wrong skip is expensive to get wrong (legal, medical, financial), bias the router toward searching when it's unsure. A false "search" costs a little money; a false "don't search" can cost a customer, a compliance finding, or worse.
- A router is a small classifier, and Doc06's "Choosing between multiple tools" selection-accuracy discipline applies directly: keep a small fixture of test questions with the expected route for each, and re-run it on every prompt or model change — "it felt right" is not a test.
- **Applied to Project 3:** the Retriever step in the DocuMind graph (Project 3's Step 3) is exactly this router plus a search node — a task like "hi, thanks!" routes straight to the Reasoner, while "what does our refund policy say about late deliveries?" routes through Retrieve first. The router function itself is a few lines; the discipline around testing it both ways is the actual work.

| Situation | What to do | Why | Example |
|---|---|---|---|
| Most tasks are about your documents | Always search (plain RAG, Doc08) | A router adds cost and a new way to be wrong, for almost no gain | An HR-policy bot where every question is about policy |
| Tasks are a real mix of "needs documents" and "doesn't" | Route with a conditional edge | Skips waste and noise on the easy tasks | A support desk: "reset my password" vs. "what is your refund rule?" |
| A wrong skip is very costly (legal, medical, money) | Route, but lean toward "search" when the router is unsure | A missed search gives a made-up answer; an extra search only costs a little | `return "retrieve" if confidence < 0.8 else "answer"` |
| You cannot yet tell the two kinds of tasks | Start with always-search, log the tasks, | You need real examples before you can write a | Keep a log of 100 real questions first |

| Situation | What to do | Why | Example |
|---|---|---|---|
| apart | add routing later | good rule or train a good router | |

```
# Approach 1 — rules: cheapest, most predictable, misses unusual wording
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict, total=False):
    task: str
    found_text: str
    answer: str

DOC_WORDS = ("policy", "refund", "warranty", "contract", "handbook")

def route_after_start(state: AgentState) -> str:
    task = state["task"].lower()
    for word in DOC_WORDS:
        if word in task:
            return "retrieve"
    return "answer"

builder = StateGraph(AgentState)
builder.add_node("retrieve", retrieve_node)
builder.add_node("answer", answer_node)
builder.add_conditional_edges(START, route_after_start, ["retrieve", "answer"])
builder.add_edge("retrieve", "answer")
builder.add_edge("answer", END)
graph = builder.compile()
```

```
# Approach 2 — a small model call: handles wording rules can't predict
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from logging_setup import get_logger              # Doc01's get_logger(name)

logger = get_logger(__name__)

class SearchDecision(BaseModel):
    needs_search: bool
    reason: str

router_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0
).with_structured_output(SearchDecision)
```

```
def route_after_start(state: AgentState) -> str:
    question = (
        "Does this task need our internal documents? "
        f"Task: {state['task']}"
    )
    decision = router_model.invoke(question)
    logger.info(
        "route: needs_search=%s reason=%s",
        decision.needs_search,
        decision.reason,
    )
    if decision.needs_search:
        return "retrieve"
    return "answer"
```

Approach 1 costs nothing and is instant, but only ever catches the exact words you thought of. Approach 2 catches paraphrased questions rules would miss, at the cost of one small, cheap model call and one more thing that can be wrong. Approach 3 — search as a tool the main model chooses for itself — is covered with its own code in Combining retrieval with tool calls.

**Common mistakes:**

- *Mistake:* a router that silently skips search for a task that really needed it. → *Symptom:* the graph finishes normally and the answer *looks* fine, but it's built on the model's general knowledge, not your documents — a silent wrong answer, not a crash. → *Fix:* log the routing decision on every run, and keep a small "must search" test set; if any of them shows `route=answer`, the router is too strict.
- *Mistake:* using the same expensive main model to decide whether to search. → *Symptom:* every task pays for two full model calls before any real work starts, and the routing decision itself becomes non-repeatable if temperature isn't pinned. → *Fix:* route with plain rules or a small, cheap model at `temperature=0`; save the expensive model for the real answer.

**Where you'll meet it:** this is Doc09's conditional edge mechanism, applied to the very first real decision this document makes. This document's Intermediate exercise (`conditional_search`) and Project 3's Retriever step (Step 3) are this router for real. In Doc11, a **supervisor** agent asks the exact same shape of question one level up — "which agent should handle this?" instead of "should I search?" — and the routing function you write here is the small version of the supervisor router in Project 4 and Project 11.

**Quick cheat sheet:**

- A router is a plain function (or a small model call) that returns the name of the next node — never the expensive main model.
- Pass the explicit list of possible node names to `add_conditional_edges` so a typo fails at `compile()`.

- Start with rules; move to a small model call only when rules miss real questions; let the main model choose via a tool only in an already-agentic system.
- When a wrong skip is costly, lean toward searching.
- Log every routing decision, and test both paths (search and no search) as a real fixture, not a vibe.

## Why "agentic RAG" is really a graph problem, not a search problem

Plain RAG (Doc08) is a fixed pipeline: always search, then always answer from whatever came back. Agentic RAG makes search a *decision* the running program makes, not a step that always fires — it can skip search entirely, search again with a better question when the first try comes back weak, or take a separate "I genuinely can't answer this" path when nothing relevant exists. Picture two librarians. The first always hands you the closest book on the shelf and says "here you go," whether or not it actually answers your question. The second reads your question, checks what she found, and if it's not right, goes and looks again with a better search term — and if she still can't find anything, tells you honestly instead of handing over a book that only looks related. Agentic RAG is the second librarian, and the "how do I know when to search again, or give up" logic is exactly what a graph is for.

### How it really works

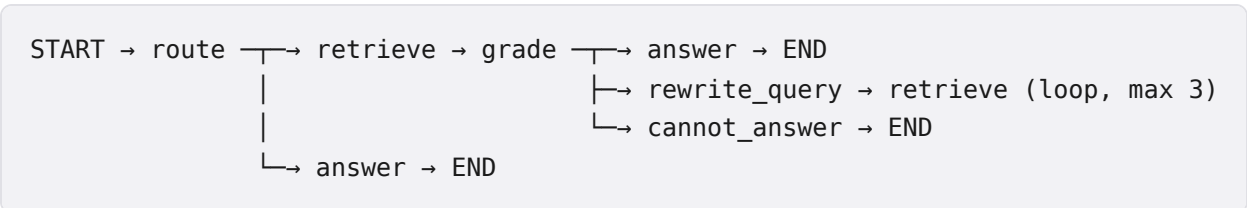
- **What "agentic RAG" is:** a graph (Doc09) where search, grading the results, rewriting the query, and answering are each their own node, connected by conditional edges instead of one straight pipeline.
- **Why it exists:** the hard part was never the search call itself — Doc08's `retrieve()` already works. The hard part is the *control flow*: which step runs next, what happens on a weak result, and when to stop. A graph is the tool built for exactly that kind of decision, the same way Doc09's LangChain vs. LangGraph vs. RAG topic explains RAG is a technique that becomes one node, not a competing framework.
- **When to reach for it:** only once plain RAG's single straight path genuinely isn't enough — when results are often weak or missing, when an action downstream needs human review, or when a question sometimes needs a second, better-informed search. Reaching for it before that point is complexity with no payoff; see the comparison table below.
- **Why this wasn't possible back in Doc08:** it needs Doc09's conditional edges and typed state. Written as plain functions, "if the result is weak, rewrite the query and try again, but at most 3 times, and if still nothing, say 'I don't know'" turns into nested `if` / `while` code that is hard to read, test, or pause. In a graph, each of those choices is one small, named, testable edge.
- **The grade step is the genuinely new idea**, and it itself has two real implementations, cheapest to most reliable:
- **A score threshold. What:** compare the best match's score — Doc08's distance, where lower means closer — (Doc08's Vector stores topic) against a fixed cutoff. **Why:** free — no extra model call, uses a number you already have. **When:** your embedding model's scores are well-

calibrated for your documents and you've actually checked the cutoff against real "good" and "bad" examples, not guessed one.

- **A small model call as judge. What:** ask a cheap model "do these chunks actually answer this question, yes or no?" with structured output. **Why:** a similarity score measures *closeness in meaning space*, not *whether the text actually answers the question* — a chunk can be the closest thing in the store and still miss the point. **When:** similarity scores keep passing chunks that don't actually help, or the cost of one small judge call is worth the extra reliability.
- The loop needs a real stopping rule, or it is just Doc07's step-limit discipline rebuilt with worse code. `MAX_SEARCHES` in state, checked by the router *before* looping, not after — the same "layer real limits, don't trust judgment alone" rule.
- `found_chunks` accumulates across every search attempt in the loop, not just the last one — the answer often needs facts the first hop found *and* facts the second, more specific hop found. Replacing instead of appending silently throws away real, useful, already-paid-for context.
- **Applied to Project 3:** the graph shape below ( `route → retrieve → grade → answer / rewrite_query / cannot_answer` ) is exactly what Project 3's Retriever agent (Step 3) implements — it searches, judges its own results with an explicit relevance rule written down in advance (not a vague "does this look okay?"), and routes to a "can't ground this" path instead of forcing the Reasoner to answer from nothing.

**Plain RAG vs. agentic RAG, side by side:**

| Question | Plain RAG (Doc08) | Agentic RAG (this document) |
|---|---|---|
| Does it always search? | Yes | Only when the router says so |
| What if the results are weak? | Answers anyway (risk of a made-up answer) | Rewrites the query and searches again, or stops |
| What if nothing is found? | Usually still answers | Goes to an explicit "can't answer" node |
| Can a human check before an action? | No | Yes, with `interrupt()` |
| Cost per question | Fixed: 1 search + 1 model call | Changes: 0 to N searches + a grading check |
| How easy to test? | Very easy: one straight path | Each path needs its own test |



```
# Approach 1 — grade with the vector store's own scores (cheap)
# Doc08's score is a DISTANCE: the best match has the SMALLEST score
def grade_by_score(chunks: list[dict], max_distance: float = 1.2) -> bool:
    if not chunks:
        return False
    best = chunks[0]["score"]
    for c in chunks:
        if c["score"] < best:
            best = c["score"]
    return best <= max_distance

# Approach 2 — grade with a small model call (more reliable, costs a call)
class ResultsGrade(BaseModel):
    good_enough: bool
    reason: str

grader_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0
).with_structured_output(ResultsGrade)

def grade_by_model(question: str, chunks: list[dict]) -> bool:
    pieces = []
    for c in chunks:
        pieces.append(c["text"])
    text = "\n\n".join(pieces)
    prompt = (
        f"Question: {question}\n\nFound text:\n{text}\n\n"
        "Does this text actually answer the question?"
    )
    grade = grader_model.invoke(prompt)
    return grade.good_enough

MAX_SEARCHES = 3

def route_after_grade(state: AgentState) -> str:
    if state["results_are_good"]:
        return "answer"
    if state["search_count"] >= MAX_SEARCHES:
        return "cannot_answer"
    return "rewrite_query"

builder.add_conditional_edges("grade", route_after_grade,
                              ["answer", "rewrite_query", "cannot_answer"])
```

**Common mistakes:**

- *Mistake:* building agentic RAG with no "can't answer" path — every route after `retrieve` eventually reaches `answer`. → *Symptom:* the graph searches, finds weak results, and the answer node answers anyway — a confident, made-up answer that's worse than plain RAG because it *looks* checked. → *Fix:* draw the graph with `graph.get_graph().draw_mermaid()` (Doc09's debugging-a-graph-visually topic); if every path ends in `answer`, add the missing exit.
- *Mistake:* grading with a score cutoff picked by guessing, never checked against real examples. → *Symptom:* the grader passes chunks that don't actually answer the question, or rejects chunks that do, and nobody can explain why the threshold is `0.75` and not `0.6`. → *Fix:* collect real "good" and "bad" examples first and pick the threshold from them, or switch to a small model-call grader if scores keep disagreeing with what a person would judge.

**Where you'll meet it:** this document's Edge-case exercise (`empty_search`) is the `cannot_answer` path in miniature. Project 3 is a full agentic RAG graph, built directly on Doc09's state, conditional edges, and loop mechanics. In Doc11 and Project 4, a "research agent" is often this whole graph wrapped as one node — a subgraph, exactly Doc09's Subgraphs topic — inside a bigger multi-agent graph. Project 12 shows the same idea behind an MCP tool. Doc13 tests each of these paths separately.

**Quick cheat sheet:**

- Plain RAG is a straight line; agentic RAG is a graph with choices and loops — the same control-flow tools Doc09 already gave you.
- The new pieces are: the router, the grader, the rewrite loop, and the "can't answer" exit.
- Grade with a score cutoff (cheap) or a small model judge (more reliable) — pick based on how well your scores are actually calibrated.
- Every loop needs a hard limit (`MAX_SEARCHES` in state), checked before looping, the same discipline as Doc07's step limit.
- Use plain RAG when it's good enough — it's cheaper and easier to test. Test every path, not just the happy one.

## State must carry what was found, not just what was asked

When a human-approval pause happens after search, the state saved at that pause must include the found text itself, not only the original question. On resume, the graph should continue with exactly the text a person already looked at — never search again to "fill in" what should have been saved. Think of a doctor's patient file — Doc09's own analogy for state: if the file only records "patient came in complaining of pain" and not what the last test actually showed, every new doctor who opens the file has to re-run the test before doing anything, even if the original test result would have been perfectly fine to act on. A file that only records the question, not the answer, forces everyone downstream to redo work that was already done.

**How it really works**

- **What this is:** designing the graph's `TypedDict` (or Pydantic model) state shape, in Doc09's State sense, so that everything a later node or a human approver needs is actually present in it — not just the task, but the found chunks, the draft answer, and a running log.
- **Why it matters specifically here:** the checkpointer (Doc09) saves the *whole state dict* after every step, for free, with no extra code. If you never put the found text into state, there is nothing for the checkpointer to save, and resuming after a pause means either re-searching (see below) or the resumed run continuing with a gap nobody planned for.
- **When under-saving state actually bites:** three concrete ways. First, the documents can genuinely change during a pause of minutes or hours, so a fresh search on resume can return different text than what a human approved — the final answer is now based on text nobody actually checked. Second, you pay for the same search twice for no reason. Third, if the vector store happens to be down at the exact moment of resume, a run that was *already approved* now fails for a reason that has nothing to do with the approval.
- **`TypedDict` vs. a Pydantic model for state — two real choices, not one right answer:**
- **`TypedDict`. What:** a plain typing construct with zero runtime behavior — just type hints your editor and `mypy` check. **Why:** matches LangGraph's own examples and prebuilt nodes exactly (`ToolNode`, `tools_condition` expect a plain dict at runtime regardless), and stays a plain dict you can print, merge, and pass around with no extra machinery. **When:** the default choice for most graphs, including this document's — nothing here needs runtime validation on every node's output.
- **A Pydantic model. What:** a real class with `__init__` validation — an update that puts a `str` where you declared an `int` raises immediately instead of silently passing through. **Why:** worth it when a field's *shape* genuinely matters and a wrong shape from one node would otherwise corrupt a later node's assumptions with no warning. **When:** a graph with many contributors, or one where a bad update has caused real, hard-to-trace bugs before — the validation cost is worth paying only once it has actually been earned.
- A node never edits state in place — it **returns** a small dict describing the change, and the engine merges it using each field's reducer, exactly as Doc09's State topic requires. `Annotated[list[str], operator.add]` is the reducer used here for `path_log` — a node returning `{"path_log": ["searched"]}` *appends* to the log instead of replacing it, so the full trail of what happened survives to the end.
- **Applied to Project 3:** the `AgentState` below is close to what Project 3's `state.py` actually needs by Step 5 — `found_chunks` (with each chunk's source and score, so the approver and the final answer can both show where the text came from), `draft_answer`, `decision`, `search_count` for the loop limit, and `path_log` for a readable trail of what the graph actually did.

```
from typing import Annotated, TypedDict
import operator

class AgentState(TypedDict, total=False):
    task: str                    # what was asked
```

```
    # the search text actually used, may be rewritten
    query: str
    found_chunks: list[dict]     # what was found: text + source + score
    draft_answer: str            # what the model wrote from those chunks
    decision: str                # "approved" / "rejected", filled on resume
    search_count: int            # for the loop limit
    path_log: Annotated[list[str], operator.add]
    # ^ "searched", "approved"...  adds up, never overwritten
```

| Situation | What to do | Why | Example |
|---|---|---|---|
| Later steps or the approver need it | Put it in state | It survives the pause and the approver's view matches the final answer | `found_chunks`, `draft_answer` |
| The source of each chunk | Keep it next to the text | The approver and the final answer can show where the text came from | `{"text": ..., "source": "refund_policy.pdf", "page": 4}` |
| A big object that can't be saved (DB connection, client, open file) | Do NOT put it in state; create it inside the node | The checkpointer must save state as plain data; these objects break saving | Make `OpenAI()` inside the node or at module level |
| Very large content (whole PDFs, images) | Store it elsewhere; keep an id or path in state | Every checkpoint copies the state; big state makes every step slow | `"doc_ids": ["a1", "b7"]` instead of the full files |
| Secrets (API keys, passwords) | Never put them in state | Checkpoints are written to a database and may be seen in logs and tools | Read keys from `.env` via Doc01's config |

```
from langgraph.types import interrupt

def approval_node(state: AgentState) -> dict:
    sources = []
    for c in state["found_chunks"]:
        sources.append(c["source"])
    decision = interrupt({
        "draft": state["draft_answer"],
        "sources": sources,
    })
    return {"decision": decision, "path_log": [f"approval:{decision}"]}
```

```
def final_node(state: AgentState) -> dict:
    # Uses the SAME found_chunks the human saw. No new search.
    if state["decision"] != "approved":
        return {"answer": "Stopped: not approved.", "path_log": ["stopped"]}
    return {"answer": state["draft_answer"], "path_log": ["finished"]}
```

**Common mistakes:**

- *Mistake:* doing the search inside the same node that calls `interrupt()`. → *Symptom:* on resume, LangGraph re-runs that node **from its first line**, not from the `interrupt()` line, so the search fires a second time and may return different results than what the person actually approved. → *Fix:* put search in its own node, before a separate approval node — everything before `interrupt()` in that node's own body must be safe to repeat, and re-searching isn't.
- *Mistake:* saving only the original question in state, assuming resume can just "search again" for what's needed. → *Symptom:* a resumed run's final answer doesn't match what the approver actually saw, or fails outright if the vector store happens to be briefly down at resume time. → *Fix:* put the found chunks, with sources, directly in state — the checkpointer saves them for free once they're there.

**Where you'll meet it:** this document's Real-world exercise (`approval_pause`) and Failure exercise (`search_failure`) are exactly this, and Project 3's `state.py` is this shape grown by one field per step. Doc09's Checkpointers and `interrupt()` topics are the mechanics this whole topic depends on. In Doc11, **shared state design** is this exact question for many agents at once — a research agent puts its findings in state so the writer agent never has to search again.

**Quick cheat sheet:**

- Save the found text (with its sources) in state, not just the question.
- On resume, use the saved text; re-search only on purpose, and show the new text again if you do.
- Keep search and `interrupt()` in separate nodes — an interrupted node re-runs from the top on resume.
- `TypedDict` by default; reach for a Pydantic state model only once a bad shape has actually caused a real bug.
- No clients, connections or secrets in state; big files go elsewhere, with an id in state; use `operator.add` for logs that should grow.

### Multi-step search: when one search isn't enough

This document's basic exercises use one search-then-answer flow: decide if search is needed, search once, answer from what was found. Multi-step search is for questions where the first search genuinely is not enough, and only reading its results tells you what a *better* second search would even look like. "What was the reason for the delay mentioned in the March report?" needs one search to find the March report and learn what the delay was about — say, "customs at the port" —

and only then can a second, more specific search find the real reason. A single search using the whole original question often returns generic text about "delays" and misses the actual answer. This is the graph-shaped version of a two-hop question, and without it the agent either answers from weak text or gives up too early.

### How it really works

- **What it is:** instead of a straight line from search to answer, the routing function after search checks whether the results are good enough. If not, it sends the graph to a `refine_query` node and back to `retrieve` — Doc09's "loop on purpose" pattern, with a real stopping rule.
- **Why it exists:** some real questions are two hops, and no amount of clever single-shot phrasing fixes that — the second query genuinely needs a fact (a name, a code, a date) that only exists inside the first result.
- **When to reach for it:** only when you've actually seen questions like this in real use — "the X mentioned in Y" shaped questions, or first-attempt searches that reliably come back close-but-not-quite. Adding this loop to a system where every question is single-hop is pure overhead with nothing to show for it.
- **Two ways to decide "good enough," the same two options as the grading step above, reused here on purpose because it's the same judgment call:** a score cutoff (cheap, needs a calibrated threshold) or a small model-judge call ( `good_enough(chunks, task)` , more reliable, costs one more call per loop). Whichever you pick for the grading topic above, use consistently here too — two different graders answering the same question differently is confusing to debug.
- `found_chunks` must accumulate across *every* search in the loop, not just the last one — the final answer often needs a fact from the first hop and a fact from the second hop together. Replacing the list on each retrieve silently throws away the first hop's finding the moment the second hop runs.
- `search_count` is checked **before** looping, inside the routing function, not inside the node doing the search — the stopping rule has to live in the edge that decides whether to loop again, the same place Doc09's Loops topic puts it.
- A refine step that keeps writing nearly the same query each time ("delay reason March", "March delay reason") burns the whole search budget and lands on `cannot_answer` having learned nothing new. Give the refine prompt what was already found *and* what's still missing, not just the original question again.
- **Applied to Project 3:** the multi-step loop is the natural extension once Step 3's Retriever agent needs to handle a two-hop question inside your own document set — the same `retrieve → grade → refine_query → retrieve` shape, just with `MAX_SEARCHES` capping it, exactly as shown below.

| Situation | What to do | Why | Example |
|---|---|---|---|
| The question has one clear topic | One search | A second search adds delay and cost for nothing | "What is our leave policy for new staff?" |

| Situation | What to do | Why | Example |
|---|---|---|---|
| The question links two facts ("the X mentioned in Y") | Multi-step: find Y first, then search for X inside it | The second query needs words only the first result has | "Who approved the vendor named in the Q2 audit?" |
| First results are close but miss a key word | One refine step with a rewritten query | Different wording often finds the right chunk | User says "laptop"; documents say "notebook computer" |
| Nothing relevant exists in your documents | Stop after the limit and go to `cannot_answer` | More searches will not create missing information | A question about a product you don't sell |
| A strict time limit (live chat) | Limit to 1-2 searches, or skip refining entirely | Each loop adds seconds the user waits | `MAX_SEARCHES = 2` |

```
MAX_SEARCHES = 3

def refine_query_node(state: AgentState) -> dict:
    prompt = (
        f"Original question: {state['task']}\n"
        f"Last search text: {state['query']}\n"
        f"What we found so far: {summarize(state['found_chunks'])}\n"
        "Write ONE better search query to find the missing information."
    )
    new_query = model.invoke(prompt).content.strip()
    return {"query": new_query, "path_log": [f"refined:{new_query}"]}

def retrieve_node(state: AgentState) -> dict:
    chunks = retrieve(collection, state["query"], k=4)
    earlier = state.get("found_chunks", [])          # keep earlier findings too
    return {
        "found_chunks": earlier + chunks,
        "search_count": state.get("search_count", 0) + 1,
    }

def route_after_retrieve(state: AgentState) -> str:
    if good_enough(state["found_chunks"], state["task"]):
        return "answer"
    if state["search_count"] >= MAX_SEARCHES:
        return "cannot_answer"
    return "refine_query"
```

**Common mistakes:**

- *Mistake:* checking `search_count` inside the `retrieve` node instead of inside the routing function. → *Symptom:* the loop limit is inconsistently applied, or a code path that skips the retrieve node's own check loops anyway. → *Fix:* the stopping rule belongs in the conditional edge that decides whether to loop again — one place, always checked, exactly as Doc07's step-limit topic and Doc09's Loops topic both require.
- *Mistake:* a refine step that writes almost the same query every time. → *Symptom:* the loop uses its whole search budget, finds the same chunks each time, and lands on `cannot_answer` having paid for three searches for nothing. → *Fix:* log every query in `path_log` and read them; give the refine prompt what's already found and what's still missing, and stop early if a new search returns the same chunk ids as the last one.

**Where you'll meet it:** this extends this document's `empty_search` exercise, and Project 3 if you add a retry path to the Retriever agent. In Doc11 and Project 4, the research agent usually does multi-step search on its own before handing findings to the writer, and the **generator → critic → revision** loop there uses the identical "check, improve, loop with a limit" shape.

**Quick cheat sheet:**

- Use multi-step search only when the second query genuinely needs a fact from the first result.
- The loop is `retrieve → grade → refine_query → retrieve`, with a counter in state, checked in the routing function.
- Grade with a score cutoff or a small model judge — the same choice as the grading step, applied consistently.
- Always set a hard `MAX_SEARCHES` and route to `cannot_answer` when it's reached.
- Keep findings from every search in the loop, not just the last one; log every query so you can see if refining is actually improving anything.

## Combining retrieval with tool calls

It's easy to think of "search my documents" (Doc08's RAG) and "call a tool" (Doc06's tool calling) as two different things, because they were taught in two different documents. They aren't. A retriever is just a function that takes a query and returns text — which is exactly what a tool is. Once your document search is wrapped the same way as a weather API or a calculator, the model can choose, inside the same reasoning loop, to call "search my documents," then "check this order's status," then "search my documents again" — all ordinary tool calls, decided by the same model, in the same turn-by-turn loop Doc07 already built. Without this, you end up maintaining two separate systems — one for RAG, one for tools — plus glue code to decide which one to use when, for no real benefit.

**How it really works**

- **What it is:** `retrieve()` wrapped with `@tool` (Doc06), exactly like any other function, then bound alongside your other tools with `bind_tools([...])`.
- **Why it exists:** a real task rarely needs *only* documents or *only* live data — "is my order late, and what does the delivery policy say?" needs both, and forcing them through two separate code

paths means writing your own logic to decide which one runs when, which the model's own tool-choice already solves for free.

- **When to reach for it:** whenever the model needs to freely mix document facts with live data, in an order it decides based on what it's already found — the exact case Doc07's ReAct loop was built for.
- The tool must return **text** — a `str` — because the model reads the result as text, the same shape rule as every other tool result. Doc08's `retrieve()` returns a list of chunk dicts, so the tool formats them, and it includes the source so the model can cite it, and returns a clear `"NO_RESULTS: ..."` string on an empty match rather than an empty string a model might misread as "found nothing worth mentioning" versus "the tool broke."
- **There are genuinely two ways to run this, and picking between them is a real trade-off, not a stylistic choice:**
- `create_agent` **(the ready-made loop). What:** `create_agent(model, tools=[search_documents, check_order_status], system_prompt="...")` — Doc07's exact library shortcut. **Why:** shortest correct code; it already compiles to a LangGraph graph underneath (Doc07's ReAct loop topic and Doc09's tool-calling-inside-a-graph topic explain exactly what it's doing for you). **When:** the model deciding everything is fine — no routing rules, no loop-count customization, no approval gate needed.
- **Your own graph. What:** a model node with `bind_tools`, a `ToolNode` (from `langgraph.prebuilt`), and a conditional edge looping back while the model asks for tools — you build the same loop by hand, one node at a time. **Why:** you need to own the edges — to insert a node between "the model wants to act" and "the action runs," a hook `create_agent` doesn't expose. **When:** you need routing rules, a custom loop limit, or — the case this whole document is really about — an approval step before a risky action (Project 3's exact shape).
- **A third, related choice, worth naming explicitly:** search does not have to be optional. If a task category must *always* be grounded before an approval gate — a compliance answer that must quote the policy — make search a **fixed graph node**, not a tool the model might skip. A tool is a choice the model can decline; a node in a fixed edge sequence always runs. Doc06's "tool description is really a prompt" topic explains why a model can be talked out of using a tool it was merely offered — a fixed node has no such escape hatch.
- With many tools (10+), the docstring is the only thing steering tool choice — Doc06's "tool description is a prompt" topic applies directly: say clearly when to use it and, just as importantly, when **not** to.
- **Applied to Project 3:** Project 3's own Retriever step is deliberately built as the "your own graph" approach, not `create_agent` — because Step 5's Approval gate needs a dedicated node between the Reasoner's draft and the actual action, exactly the hook `create_agent` doesn't give you.

| Way | Code | Good for |
|---|---|---|
| **Ready-made agent loop** | `from langchain.agents import create_agent` then `agent = create_agent(model, tools=` | Quick start; the model decides |

| Way | Code | Good for |
|---|---|---|
| | `[search_documents, check_order_status], system_prompt="...")` | everything |
| **Your own graph** | A model node with `bind_tools`, a `ToolNode` (`langgraph.prebuilt`), and a conditional edge that loops back while the model asks for tools | Routing rules, loop limits, or an approval step (Project 3) |
| **A fixed graph node** (not a tool at all) | Search runs unconditionally, as a normal node in a fixed edge sequence | Search that must *always* happen before an approval gate |

```
from langchain.tools import tool
from langchain_openai import ChatOpenAI

@tool
def search_documents(query: str) -> str:
    """Search the company knowledge base (policies, manuals, FAQs).
    Use this for questions about company rules or products.
    Do NOT use it for order status — use check_order_status for that."""
    chunks = retrieve(collection, query, k=3)
    if not chunks:
        return "NO_RESULTS: nothing in the knowledge base matched this query."
    pieces = []
    for c in chunks:
        pieces.append(f"[{c['source_file']}] {c['text']}")
    return "\n\n".join(pieces)

@tool
def check_order_status(order_id: str) -> str:
    """Return the current status of one order, by order id like 'A-1042'."""
    # Doc02's http_client already retries
    return orders_api.get_status(order_id)

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model_with_tools = model.bind_tools([search_documents, check_order_status])
```

**Common mistakes:**

- *Mistake:* a vague tool docstring, like `"""Search."""`. → *Symptom:* the model can't tell when to use it, so it either searches for everything — including questions your documents can't answer — or almost never searches. → *Fix:* log which tools the model calls across 10-20 test questions; if the wrong tool is chosen often, rewrite the docstrings first, before touching the prompt or the model.

- *Mistake:* making search an optional tool for a task category where it must always run before an approval gate. → *Symptom:* the model occasionally answers a compliance-sensitive question without ever calling `search_documents`, because a tool is a suggestion it can decline. → *Fix:* make that search a fixed graph node in the edge sequence, not a tool — a node always runs; a tool might not.

**Where you'll meet it:** this document's Basic exercise (`search_as_tool`) is this exact wiring, and Project 3 builds the "your own graph" version of it for real. In Project 7 and Project 8, this same search function is shared through MCP so any agent can call it. In Doc11 and Project 11, each specialist agent gets only the tools its job needs — only the research agent gets `search_documents`. Project 9 is why a search tool's returned text must be treated as data, never instructions, exactly as Doc06's prompt-injection topic warns.

**Quick cheat sheet:**

- A retriever is just a tool: a function with a query in and text out, same shape as any other tool.
- The tool returns a `str`, with sources, and a clear "no results" message, never a silent empty string.
- The docstring says when to use the tool, and just as clearly, when not to.
- `create_agent` for a quick loop; your own graph when you need routing, limits, or an approval step; a fixed node when search must never be skippable.
- If search must always happen, it's a node, not a tool a model might decline.

## Caching search results within one run

A cache is a place you keep a result you already got, so you don't have to get it again. Here the cache lives only for one graph run: if the graph searches for the same query twice in one run, the second time it reuses the saved result instead of asking the vector store again. If your graph can pass back through the search node more than once — from the multi-step loop above, or from a model calling the search tool again on its own — it can easily run the *identical* search twice. That means paying for the same embedding call twice and waiting on the vector store twice, and inside one run, spanning a few seconds or minutes, the documents haven't changed — the second search buys you nothing at all.

**How it really works**

- **What it is:** a plain Python dictionary mapping a normalized query string to its results, kept as a field in the graph's state so it survives the run (and, because it's in state, survives a pause too).
- **Why it exists:** any loop that can revisit the search node — the multi-step search loop, or a model freely calling a search tool more than once — can genuinely ask the identical question twice in one run, and a per-run cache turns that into a free lookup instead of a second paid call.
- **When to reach for it:** only once your graph can actually loop back to search within a single run. A search-once-then-answer flow has nothing to cache — skip it there; it's pure state and code for zero saving.

- **Two real places to put the cache, depending on which of the two search approaches above you're using:**
- **As a state field, for the graph-node version. What:** `state["search_cache"]`, a plain dict, checked before calling `retrieve()`. **Why:** a node reads and returns state exactly like any other field — no new machinery needed, and the checkpointer saves it automatically like everything else in state. **When:** you built search as a fixed or conditional graph node (the "your own graph" approach above).
- **As a plain closure variable, for the tool version. What:** a small `dict` created fresh per run, captured by a tool-maker function (`make_search_tool(cache)`), *not* stored in graph state. **Why:** a `@tool` function can't easily read graph state — it only sees the arguments the model passed it — so the cache has to live somewhere else the tool closure can reach. **When:** search is a tool the model calls directly (the `create_agent`/tool-calling approach above); changing this dict in place is fine here, because it's a normal Python object you own, not LangGraph state.
- A node must **return a new dict**, never mutate the cache in place — `{**cache, key: chunks}`, not `cache[key] = chunks`. Doc09's State topic explains why: a node that mutates state in place can make a saved checkpoint and the live state quietly disagree, which is a very confusing thing to debug days later.
- `normalize()` matters more than it looks — "Refund Policy", "refund policy", and " refund policy " should all hit the same cache entry. Skipping it means near-identical queries all pay full price separately.
- The cache key must include **everything** that changes the result, not just the query text — the user (if different users see different documents), their permissions, the document version, and `k`. A cache keyed only on the query text in a multi-tenant system is a data leak waiting to happen: user B gets user A's private search results back from the cache.
- Never apply this same caching idea to genuinely live data — order status, stock levels, prices — the same way you cache document search. Documents are stable across a few minutes; a price is not, and `check_order_status` should always be called live, uncached, full stop.
- **Applied to Project 3:** the multi-step loop from the previous topic is exactly where this pays off in Project 3 — if the Retriever agent's refine loop happens to search for something close to an earlier query in the same run, the state-based cache above avoids a second, redundant vector-store call.

| Situation | What to do | Why | Example |
|---|---|---|---|
| The graph can loop back to search in the same run | Cache in state, keyed by the normalized query | Stops paying twice for the same search | Multi-step search, or a model calling the tool again |
| Data changes quickly (stock levels, prices, order status) | Do NOT cache, or cache only for a few seconds | An old answer can be wrong a minute later | `check_order_status` should always be called live |
| Many *different* users ask the same | A shared cache outside the graph | A per-run cache is empty for every | Many users asking "what are your opening hours?" |

| Situation | What to do | Why | Example |
|---|---|---|---|
| question | (Redis, with an expiry) | new run | |
| The graph searches only once per run | Skip the cache | It adds code and state for no saving | A simple search-then-answer flow |
| Results are very large | Cache chunk ids, or only a few top chunks | The cache is saved in every checkpoint | Save `[chunk_id, score]` instead of full text |

```
def normalize(query: str) -> str:
    # "Refund  Policy " → "refund policy"
    return " ".join(query.lower().split())

# graph-node version — cache lives in state
def retrieve_node(state: AgentState) -> dict:
    key = normalize(state["query"])
    cache = state.get("search_cache", {})
    if key in cache:
        logger.info(f"search cache hit: {key}")
        return {"found_chunks": cache[key], "path_log": ["search:cache_hit"]}

    chunks = retrieve(collection, state["query"], k=4)
    # a NEW dict, don't mutate the old one
    new_cache = dict(cache)
    new_cache[key] = chunks
    return {
        "found_chunks": chunks,
        "search_cache": new_cache,
        "path_log": ["search:called"],
    }

# tool version — cache lives in a closure, not in graph state
def make_search_tool(cache: dict):
    @tool
    def search_documents(query: str) -> str:
        """Search the company knowledge base."""
        key = normalize(query)
        if key not in cache:
            cache[key] = format_chunks(retrieve(collection, query, k=3))
        return cache[key]
    return search_documents

# new, empty cache for this one run
run_cache: dict[str, str] = {}
tools = [make_search_tool(run_cache), check_order_status]
```

**Common mistakes:**

- *Mistake:* a cache key built from the query text alone, in a system where different users can see different documents. → *Symptom:* user B's search silently returns user A's private results from the cache. → *Fix:* the key must include everything that changes the result — user, permissions, document version, `k` — or the cache must be scoped strictly per run and per user.
- *Mistake:* mutating the cache dict in place inside a node (`cache[key] = chunks`) instead of returning a new one. → *Symptom:* the live state and a saved checkpoint quietly disagree after a pause/resume, in a way that's hard to reproduce. → *Fix:* always `return {**cache, key: chunks}` — a new dict, never an in-place edit — the same rule Doc09 gives for every state field.

**Where you'll meet it:** the multi-step search loop earlier in this document, and Project 3, are exactly where a per-run cache pays for itself. In Doc11, several agents that may ask the same question share the same real-money savings. Doc12 and Project 5 move to a shared cache with an expiry time, for many users across many runs.

**Quick cheat sheet:**

- A per-run cache is a dict, from normalized query to results — in state for a graph node, in a closure for a tool.
- Check the cache first; call the real retriever only on a miss.
- Return a new dict from the node; never mutate state in place.
- Never cache live data (order status, prices) the same way as stable documents.
- The key must include everything that changes the result — query, user, permissions, `k`.

### When human approval should block vs. just notify

This document's `approval_pause` exercise uses `interrupt()` the expected way: the graph stops completely and waits for a person to say yes or no. That's the right call for some actions, but not for all of them, and treating every consequential-looking action the same way is itself a design mistake. There are two real styles: **blocking approval** — the graph pauses with `interrupt()` and the action genuinely does not happen until a human says yes — and **notify but proceed** — the action happens right away, a human is told, and can check or undo it later. If every step blocks, a human becomes the bottleneck: work waits hours for approvals that never needed a human at all, and people start clicking "approve" without reading, which quietly removes the exact safety you were trying to add. If nothing ever blocks, one wrong model output can send an email to 5,000 customers or refund the wrong amount — and that can't be undone. The actual skill is choosing, for each action, which style fits, on purpose, not by habit.

**How it really works**

- **What "blocking" is:** `interrupt()` inside a dedicated node, using Doc09's `interrupt()` mechanism — the graph genuinely stops and the checkpointer keeps its state until a human calls back with `Command(resume=...)`. **Why:** the only style that actually prevents a bad action

before it happens, because nothing about a blocked node ever runs the action. **When:** the action is hard or impossible to undo *and* the damage if wrong is real — sending a message, a payment, a deletion.

- **What "notify but proceed" is:** the node runs the action immediately and separately appends an entry to an `audit_log` field (with `operator.add`, so nothing overwrites the trail) that a human reviews later, on their own schedule. **Why:** blocking every low-risk action for review turns a human into a queue for things that were always going to be approved anyway — real cost with no real safety gained. **When:** the action is easy to undo, or the damage if wrong is genuinely small.
- **How to actually decide, in two questions:** *can this be undone easily?* and *how much damage if it's wrong?* Those two questions produce four honest cells, not a single blanket rule:

| | Easy to undo | Hard or impossible to undo |
|---|---|---|
| **Small damage if wrong** | Notify but proceed | Notify, or block only above a limit |
| **Big damage if wrong** | Notify, and review soon | **Block** with `interrupt()` |

- **A third real pattern, worth naming even though this document doesn't build it: a time-boxed veto window. What:** the action is scheduled, a human is notified immediately, and it actually executes only after a fixed delay (minutes to hours) unless someone actively cancels it in that window. **Why:** it gives blocking's real safety — a human genuinely can stop it — without blocking's cost, because most actions that would have been approved anyway just run on schedule with nobody having to click anything. **When:** high-volume, mostly-safe actions where a full block would create an unworkable approval queue, but a pure notify-after-the-fact leaves too little room to catch the rare bad one — a middle ground many real payment and deployment systems actually use.
- A money limit is the most common real version of "block only above a threshold": small refunds go through automatically and are logged; refunds above a set amount wait for a manager. This turns one blanket policy into a graded one, without needing a full case-by-case judgment call on every run.
- **The one hard rule regardless of style:** never put the actual side effect *before* `interrupt()` in the same node — `send_email(draft)` then `interrupt("ok?")` sends the email before anyone approves, and worse, sends it **again** on resume, because the node re-runs from its first line (Doc09's `interrupt()` topic states this directly). The safe shape is always two nodes: an approval node containing only `interrupt()`, and a separate `do_action` node reached only when `decision == "approved"`.
- A node **returns** a small dict of changes — `decision` and a new `audit_log` entry — it never edits `state` in place, the same rule as every other topic here. `interrupt()` itself gives back whatever value was passed to `Command(resume=...)`; a node must still return a proper dict of state fields, not that raw value directly.
- **Applied to Project 3:** Step 5's Approval agent is exactly this decision, made explicit as a checklist rather than one vague model call — it runs automatic checks on the Reasoner's draft

(does it actually cite found content? does it avoid a risky action with no support?), auto-approves the clearly safe ones, and calls `interrupt()` only when its own checks aren't confident or the action is genuinely consequential.

**When to use it / when NOT to:**

| Situation | What to do | Why | Example |
|---|---|---|---|
| Sending an email or message to a customer | Block | A sent message can't be taken back | `interrupt({"email_draft": ...})` |
| Money: payments, refunds, orders | Block, or block only above a limit | Money mistakes are costly and slow to fix | Refunds under Rs. 2,000 go through; bigger ones wait |
| Deleting or overwriting records | Block | Data may be gone for good | "Delete 312 old customer records?" |
| Updating an internal draft, or adding a tag | Notify and proceed | Easy to undo; waiting adds delay for no safety gain | Tag a ticket `needs_review` and log it |
| Posting to an internal Slack channel | Notify and proceed | Low risk, and people can correct it | Log the post in `audit_log` |
| High-volume, mostly-safe scheduled actions | Notify + a time-boxed veto window | Full blocking doesn't scale; pure notify skips the rare bad case | A nightly batch of auto-generated emails, sent unless flagged within 30 minutes |
| A scheduled night job with nobody awake | Don't block; collect risky actions for a morning review queue | A blocked run would wait all night | Write "pending actions" to a table |

```
import operator
from typing import Annotated, TypedDict
from langgraph.types import interrupt, Command

class ActionState(TypedDict, total=False):
    action: dict          # {"type": "refund", "amount": 5000, "risk": "high"}
    decision: str
    audit_log: Annotated[list[dict], operator.add]  # grows, never overwritten
```

```
def approval_gate(state: ActionState) -> dict:
    action = state["action"]
    if action["risk"] == "high":
        decision = interrupt({"please_approve": action})   # pauses here
        return {
            "decision": decision,
            "audit_log": [
                {"action": action, "mode": "blocked", "decision": decision}
            ],
        }
    logger.info("auto-approved low-risk action: %s", action)   # Doc01's logger
    return {
        "decision": "approved",
        "audit_log": [{"action": action, "mode": "notified"}],
    }

def route_after_gate(state: ActionState) -> str:
    if state["decision"] == "approved":
        return "do_action"
    return "report_rejected"

# Resume later, with the SAME thread_id the run was started with:
# config = {"configurable": {"thread_id": "refund-881"}}
# graph.invoke(Command(resume="approved"), config)
```

**Common mistakes:**

- *Mistake:* a side effect before `interrupt()` in the same node — `send_email(draft)` then `interrupt("ok?")`. → *Symptom:* the email is sent before anyone approves, and sent **again** on resume, because the node re-runs from its first line. → *Fix:* an approval node containing only `interrupt()`; a separate `do_action` node runs the real effect, reached only when `decision == "approved"`.
- *Mistake:* blocking every action "to be safe," regardless of how easy it is to undo or how small the damage would be if wrong. → *Symptom:* people stop reading approval requests and start clicking "approve" on everything, which quietly removes the safety a block was supposed to add. → *Fix:* apply the undo/damage table above per action, not one blanket rule for the whole system.

**Where you'll meet it:** this document's Real-world exercise (`approval_pause`) and the Build Task's "rejected on resume" test case are exactly this. In Doc11 and Project 4, one approval gate usually sits at the *end* of the whole multi-agent pipeline, before publishing, not after every single agent. In Project 5 and Doc12, the pause becomes a real product feature: the API returns "waiting for approval," and a later request resumes the run. Doc13 uses the `audit_log` to review what the agent did on its own.

**Quick cheat sheet:**

- Block when the action is hard to undo AND costly if wrong; otherwise notify and log.
- Money often uses a limit: small amounts notify, big amounts block.
- A time-boxed veto window is a real middle option for high-volume, mostly-safe actions.
- Never put a side effect before `interrupt()` in the same node — it re-runs on resume.
- A node returns a dict of changes; keep an `audit_log` with an `operator.add` reducer; always handle "rejected" too.

## Go Deeper (Optional)

*You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.*

- LangGraph — tutorials index — the "RAG agent" / "agentic RAG" style tutorials.
- Look again at 08_rag's reading list — you're combining that search tool now, not learning new search theory.

## Practice Exercises

**Setup:** same venv as before — if it's not active, `cd 10_agent_workflows && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). Packages for this document: `pip install langgraph langchain-openai chromadb` (all already installed if you did Doc08 and Doc09).

**One-time setup for every exercise:** copy Doc08's Build Task files — `chunking.py`, `config.py`, `exceptions.py`, `ingest.py`, `logging_setup.py`, `retriever.py` — and its `docs/` folder into `practice/`, and add a two-line `kb.py` that builds the knowledge base once: `from ingest import build_vector_store` and `collection = build_vector_store("docs")`. Every exercise then searches with `retrieve(collection, query)`.

**Where your code lives:** all of it under `10_agent_workflows/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — the same convention as Doc01/02/07/09 — so one topic's growth from basic to intermediate stays visible in one file.

**The full file layout, all exercises:**

```
practice/
├── docs/                                  Doc08's coffee documents
├── chunking.py ... retriever.py           Doc08's Build Task files
├── approval_pause_practice.py             Real-world
├── empty_search_practice.py               Edge cases
├── kb.py                                  builds the knowledge base once
```

```
├── search_failure_practice.py           Failure
└── search_tool_integration_practice.py  Basic + Intermediate
                                         (two sections)
```

**Why each script exists:**

- `approval_pause_practice.py` — the exact `interrupt()` pattern the Build Task's approval gate needs, grounded in what was actually found, not just the question.
- `empty_search_practice.py` — the "can't ground this" path, so a missing answer never turns into a made-up one.
- `kb.py` — builds Doc08's knowledge base once, so every exercise searches the same documents.
- `search_failure_practice.py` — proves a mid-run search crash doesn't erase whatever the graph had already done.
- `search_tool_integration_practice.py` — search wired in as a plain tool, then routed conditionally — the two building blocks every later exercise assumes already work.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to intermediate, side by side.

**Jump to an exercise:** Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

### Basic — plug search in as a plain tool first

- **What:** add the Doc08 search tool as a plain tool your Doc09 graph can call, not yet conditionally routed.
- **Why:** connect the pieces one at a time — search-as-a-tool working, before search-as-conditionally-routed. Two problems at once is harder to debug than one.
- **When you'll hit this for real:** the very first step of wiring RAG into any agent graph.
- **How to code it:** wrap Doc08's `retrieve(collection, query)` as a `@tool` that returns the found text, give it to Doc09's Real-world graph ( `bind_tools` in `think`, `.invoke()` in `act` ), and ask a coffee question. Then add a clear "nothing found" message, a caught failure, a result cap, and a version that keeps the source files separately.
- **Save as:** `practice/search_tool_integration_practice.py`, under a `# Basic` section (this file also holds the Intermediate exercise below, in its own `# Intermediate` section).
- **Builds on:** Doc09's Real-world exercise (Project 2 rebuilt as a graph) and Doc08's `retrieve()` — same graph, same retriever, now connected.
- **Used later by:** the Intermediate exercise right below, which adds routing on top of this exact tool.

- **Stuck?** Hint 1 · Hint 2 · Show me the solution

### Intermediate — make search conditional

- **What:** a conditional edge that only routes to search when the task actually needs it, skipping it otherwise — proven on both paths.
- **Why:** this is the exact mechanism that keeps your graph from wasting a search call (and its cost) on tasks that don't need grounding.
- **When you'll hit this for real:** this document's own Build Task, and any real agent handling a mix of "needs my documents" and "doesn't" questions.
- **How to code it:** write a routing function that returns `"search"` or `"skip"`, wire it as the graph's first edge with `add_conditional_edges(START, ...)` (or the `set_conditional_entry_point` shortcut), and run one question that should search and one that shouldn't. Then try a model-call router, a hybrid of both, and log every decision.
- **Save as:** `practice/search_tool_integration_practice.py`, under an `# Intermediate` section (this file also holds the Basic exercise above, in its own `# Basic` section).
- **Builds on:** the Basic tool wiring in this same file, and Doc09's conditional-edge exercise — the same mechanism, now deciding whether to search.
- **Used later by:** Project 3's Retriever step (Step 3), which is this router for real.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

### Real-world — a real approval pause

- **What:** a human-checking pause before the graph acts on found information (like "I found X, should I go ahead?").
- **Why:** this is the actual production pattern Project 3 needs — grounding an approval step in what was *found*, not just the original question.
- **When you'll hit this for real:** Project 3's own Approval agent (Step 5), and any real agent taking a consequential action based on retrieved data.
- **How to code it:** after a draft node searches and writes a draft, call `interrupt()` with the draft and its source files, then resume with `Command(resume="approved")` — and again with `"rejected"`, in its own thread. Then resume with a who-and-when dict, and refuse a draft that waited too long (saving the draft time in the draft node, not the approval node).
- **Save as:** `practice/approval_pause_practice.py`.
- **Builds on:** Doc09's Failure exercise (a real `interrupt()` /resume cycle) and this document's State must carry what was found topic — the pause must show the person the actual found text, not just the question.
- **Used later by:** Project 3's Approval agent (Step 5) is this exact gate, for real.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

### Edge cases — search comes back empty

- **What:** a task where search finds nothing useful — check whether the graph carries on anyway, or follows a "can't answer" path.
- **Why:** this is the exact failure mode that turns into a hallucinated answer if you don't design for it — you need to see your own graph handle it correctly, or catch it not doing so.
- **When you'll hit this for real:** any question genuinely outside your knowledge base's coverage.
- **How to code it:** add a conditional edge after the search node that routes an empty result to an explicit "I don't have grounding for this" node — test it with an empty store. Then add a distance threshold (the best match is the *smallest* score) and a "partial" path for a match that only covers part of the question.
- **Save as:** `practice/empty_search_practice.py`.
- **Builds on:** the grading step from Why "agentic RAG" is really a graph problem — the `cannot_answer` exit, tested for real.
- **Used later by:** Project 3's Step 3, which requires this exact "can't ground this" path.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

### Failure — search fails mid-run

- **What:** make the search call fail or time out in the middle of a run, and check the graph handles it without losing saved state.
- **Why:** an external search failure (a down vector store, a network blip) shouldn't corrupt or lose everything the graph had already done — this is a real production requirement, not a nice-to-have.
- **When you'll hit this for real:** any real deployment where your vector store is a separate service that can be briefly unavailable.
- **How to code it:** make the search node raise on purpose, run the graph, and check with `get_state()` that the earlier node's work is still saved. Then add a retry with backoff for temporary errors only, and test every outcome with a small `FlakySearch` class that fails on demand.
- **Save as:** `practice/search_failure_practice.py`.
- **Builds on:** Doc09's Checkpointers topic and Doc02's timeout/retry discipline — a search call is still just an outbound call that needs the same failure handling.
- **Used later by:** the Build Task's requirement that a search failure never loses saved state.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

## Build Task — Project 3: LangGraph App

**Stuck on the Build Task?** Hint 1 · Hint 2 · Show me the solution

**Note:** this Build Task is not a `practice/build_task/` folder. It **is** Project 3 — DocuMind, the same way Doc07's Build Task **was** Project 2. The step-by-step build guide lives in project_3_documind_rag_agent/READING_README.md — follow that, not just the summary below.

**Goal:** a LangGraph agent with state, branching, a RAG search tool, saved state, and at least one human-approval pause before a risky action — a full search → answer → (pause for approval) → resume → final answer cycle.

**Requirements:**

- Builds on the Doc09 graph skeleton — doesn't start from scratch.
- A search node/tool wired in from Doc08, with a conditional edge deciding when it's used.
- At least one `interrupt()` for human approval before something consequential happens (the action can still be a placeholder, but the approval step must be real).
- Shows the state's progress (at least step by step) in the terminal, not just the final answer.
- Saved state survives the interrupt/resume cycle, for a real, non-trivial task.

**Inputs:** a free-text task that needs search, from your Doc08 set of documents.

**Outputs:** a final answer grounded in the found documents; a visible log of the path taken (searched or not, approved or not).

**Constraints:** must correctly handle "search found nothing useful" without making up an answer anyway.

**Builds on:** Doc09's graph skeleton ( `state.py` / `nodes.py` / `graph.py` ) and Doc07's tool-calling agent (Project 2) — copy both straight in rather than starting from scratch, then grow them with the Retriever/Reasoner/Approval nodes this document adds.

**Suggested files** (the Solution's small version; Project 3 grows the same graph in its own `src/` layout):

```
10_agent_workflows/practice/build_task/
├── docs/                the coffee documents (Doc08)
├── chunking.py ... retriever.py   Doc08's Build Task files, unchanged
├── graph.py             build_graph() -- the entry decision, the
│                        grounding check, and the approval pause
├── kb.py                builds the knowledge base once
├── main.py              runs a question, shows each step, asks for approval
├── nodes.py             the search / reason / approval nodes
├── state.py             the shared state shape
└── test_project3.py     the 5 Test Cases, approved AND rejected
```

**Why each file exists:**

- `docs/` and Doc08's files — the retrieval code stays exactly as Doc08 taught it, reused instead of rewritten.
- `graph.py` — grows Doc09's skeleton (conditional edges, a checkpointer, `interrupt()` ) instead of starting a new one.

- `kb.py` — the exercises' file: one knowledge base, built once, for every node to search.
- `main.py` — the one place a question gets run, with every step printed and the approval asked in the terminal.
- `nodes.py` — the actual search → reason → approve logic, kept apart from the wiring so each node is testable alone.
- `state.py` — one shared shape for what every node reads and writes, so all nodes agree on field names.
- `test_project3.py` — proves the approval gate handles rejection correctly too, not just the happy "approved" path.

**Functions/Components to build:**

- `nodes.py` → `route_search`, `search_node`, `has_grounding`, `reason_node`, `no_grounding_node`, `approval_node`, `finish_node`
- `graph.py` → a conditional edge function deciding when to route through search
- `main.py` → runs the graph, handles the pause/resume experience in the terminal

**Used later by:** Doc11 turns this single search → answer → approve pipeline into several cooperating agents; Project 4 and Project 11 reuse this same search-as-tool and approval-gate shape with multiple specialists instead of one.

## Expected Behavior

- Full cycle: task comes in → branching decides if search is needed → search happens if so → reasoning uses the found text → interrupt pauses before the risky action → resume continues → grounded final answer.
- A task that doesn't need search skips that path entirely and still finishes correctly.
- "Search found nothing" looks visibly different from "search succeeded" in the output/log.

## Test Cases

| Scenario | Expected |
|---|---|
| Task needing search, info found | Full cycle finishes, answer grounded in the found document |
| Task not needing search | Search path skipped, correct direct answer |
| Task needing search, nothing relevant found | Graph reports it can't ground the answer, doesn't make one up |
| Interrupt reached, approved on resume | Graph continues and finishes correctly |
| Interrupt reached, rejected on resume | Graph stops/reports cleanly, doesn't do the risky action |

## Break-It / Debug Preview

- A race between the saved-state write and a mid-run interrupt.
- A routing check that skips search on a task that actually needed it.
- Full debugging drill in 14_debugging_lab.

## Interview Topics Preview

- How this is different from Project 2 · where saved state actually lives · what human-checking costs in delay/user experience · agentic RAG vs. plain RAG.

## □ You Can Now Build Project 3

Docs 08-10 (plus the Async Python gate) are everything Project 3 needs. Go to project_3_documind_rag_agent/ and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

## Move On When

Project 3 runs a full search → answer → pause → resume → final answer cycle start to finish, in your own code. Full details: CURRICULUM.md §4.

---

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**